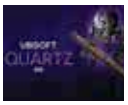
Spotify Retires Car Mode
Spotify has confirmed it's "retiring" its Simplified Car Mode. https://dgit.in/dec21-07



Ubisoft delves into NFTs
Ubisoft announced a new platform, Ubisoft Quartz, where it will offer NFTs that it's calling Digits. https://dgit.in/dec21-08

```
};
```

To update the tasks list via the database, we can run the command

```
meteor mongo
```

on the terminal, and connect to the database. Alternatively we can utilise a MongoDB GUI client, such as NoSQLBooster or Robo 3T.

The final thing we are going to explore in the tutorial is how to add new tasks via the app itself rather than directly messing with the database. Let's find out.

Creating Add Task functionality

So let's first create a TaskForm.jsx file in our UI folder, and set up an array with a string function pair as the indices. We have in imports/ui/TaskForm.jsx:

```
import React, { useState }
from 'react';

export const TaskForm = ()
=> {
  const [text, setText] =
useState("");


  return (
    <form className="task-
form">
      <input
        type="text"
        placeholder="Type to
add new tasks"
      />

      <button
type="submit">Add Task</
button>
    </form>
  );
};
```

Now, we simply need to write some logic for a submit handler. This can be done by utilising the onSubmit event as follows:

```
const handleSubmit = e => {
    e.preventDefault();

    if (!text) return;

    TasksCollection.
insert({
      text: text.trim(),
      createdAt: new
Date()
    });

    setText("");
  };
```

And finally, we hook it up to the button in React. So all in all, we have TaskForm.jsx:

```
import React, { useState }
from 'react';
import { TasksCollection }
from '/imports/api/TasksC-
ollection';

export const TaskForm = ()
=> {
  const [text, setText] =
useState("");

  const handleSubmit = e
=> {
    e.preventDefault();

    if (!text) return;

    TasksCollection.
insert({
      text: text.trim(),
      createdAt: new
Date()
    });

    setText("");
  };

  return (
    <form
className="task-form"
onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Type to
add new tasks"
        value={text}
        onChange={(e) =>
setText(e.target.value)}
      />

      <button
type="submit">Add Task</
button>
    </form>
  );
};
```

And that's it! We have successfully made a barebones To-do List App using Meteor!

CONCLUSION:

Meteor is a very simple yet powerful framework to work with, for both beginners and proffessionals. It offers a wide range of packages to assist your development process. It's simple to pick up, intuitive,

works entirely on JS, provides thorough MongoDB support, has real-time update functionality; what's there not to like? Hopefully this article gave a decent insight on how to get started with this amazing full-stack js app development platform. Good luck and happy coding!